Merkle Tree Implementation

Aztec Engineering Interview — Full Solution Document

Challenge	eng-sessions / merkle-tree (TypeScript + C++)
Depth	Configurable, default 32 ($2^{32} \approx 4.3$ billion leaves)
Hash	SHA-256 (compress: H(left ■ right), hash: H(data))
Status	✓ All 4 TypeScript tests + 4 C++ tests passing

1. Problem Overview

The challenge asks you to implement a **sparse binary Merkle tree** of configurable depth, backed by a persistent key–value store. Two identical specifications are provided:

- `eng-sessions/merkle-tree/` — TypeScript, using LevelUp / memdown (in-memory).
- `eng-sessions/merkle-tree-cpp/` — C++20, using a mock `unordered_map` database.

Both require the same three operations:

Method	TypeScript	C++
Constructor init	constructor(...)	MerkleTree(...)
Membership proof	getHashPath(index)	get_hash_path(index)
Leaf update	updateElement(index,value)	update_element(index,value)

2. Mathematical Background

2.1 Binary Merkle Trees

A **binary Merkle tree** of depth *d* is a complete binary tree with 2^d leaves, using SHA-256 as the hash function H : {0,1}* → {0,1}²■■.

- **Leaf nodes** (level 0): raw data v_i (64 bytes) → leaf hash ■_i = H(v_i).
- **Internal nodes** (levels 1…d): n_(k,p) = H(n_(k-1, 2p) ■ n_(k-1, 2p+1)).
- **Root**: n_(d, 0) — a single 32-byte commitment to all leaf data.

2.2 Tree Geometry and Bit Arithmetic

Given leaf index *i* and depth *d*:

```
Position at level k :  p_k(i) = floor( i / 2^k ) = i >> k
Left-child of pair  :  p_k(i)  &  ~1   (round down to even)
Right-child of pair :  p_k(i)  |   1   (round up to odd)
Sibling position    :  p_k(i)  ^   1   (XOR with 1)
```

2.3 Zero Hashes for Sparse Trees

With depth 32, most of the 4 billion leaf slots are never written. Rather than storing empty nodes, we pre-compute a canonical **zero hash** at each level:

```
z_0 = H( 0x00 * 64 )              // SHA-256 of 64 zero bytes
z_k = H( z_{k-1} || z_{k-1} )    // for k = 1 … d

Empty root (d=32):
  1c9a7e5ff1cf48b4ad1582d3f4e4a1004f3b20d8c5a2b71387a4254ad933ebc5
```

Any absent node at level *k* is treated as z_k without storing it — this keeps storage O(N·d) for N inserted leaves.

2.4 Hash Paths (Merkle Proofs)

For leaf index *i* in a depth-*d* tree, the **hash path** Π_i is an ordered sequence of *d* sibling pairs, one per level from the leaf upward:

```
Π_i  =  [ (n_{0, p&~1}, n_{0, p|1}),
          (n_{1, p&~1}, n_{1, p|1}),
          ...
          (n_{d-1, p&~1}, n_{d-1, p|1}) ]
where p = i >> k  at level k.
```

Verification property: Given Π_i, value v_i, and depth d, recompute c ← H(v_i); for each level-k pair (L_k, R_k) in Π_i set c ← H(L_k ■ R_k); accept iff c = root. This is exactly what Aztec's SNARK circuits verify in zero-knowledge.

2.5 Worked Example: Depth-2 Tree, 4 Leaves

Let v_i be a 64-byte buffer with i encoded in the first 4 bytes (little-endian). After inserting v_0 … v_3 into a depth-2 tree:

```
e00 = H(v0),  e01 = H(v1),  e02 = H(v2),  e03 = H(v3)
e10 = H(e00 || e01),   e11 = H(e02 || e03)
root = H(e10 || e11)

Expected root:
  e645e6b5445483a358c4d15c1923c616a0e6884906b05c196d341ece93b2de42

Hash paths:
  index 0, 1  →  [ (e00, e01), (e10, e11) ]
  index 2, 3  →  [ (e02, e03), (e10, e11) ]
```

Indices 0 and 1 share the same level-0 pair because they occupy the same two-leaf group — this is by design, as the hash path always returns the complete sibling pair at every level.

3. Algorithm Design

3.1 updateElement — Leaf Update

The algorithm walks from leaf to root in a single loop:

```
updateElement(index, value):
  c   ← H(value)        // leaf hash
  pos ← index
  batch ← []

  for level = 0 to depth-1:
    batch.add( key(level, pos) → c )   // store this node
    sibling ← getNode(level, pos ^ 1)  // DB or zero_hash[level]
    if pos is even:
      c ← H(c || sibling)              // c is left child
```

```
    else:
      c ← H(sibling || c)              // c is right child
    pos ← pos >> 1

  root ← c
  batch.add( metadata key → root )
  commit batch
  return root

Complexity: O(d) hash calls + O(d) DB reads + O(d) DB writes
```

3.2 getHashPath — Membership Proof

```
getHashPath(index):
  pairs ← []

  for level = 0 to depth-1:
    pos   ← index >> level
    left  ← getNode(level, pos & ~1)   // even sibling
    right ← getNode(level, pos |  1)   // odd  sibling
    pairs.append( (left, right) )

  return HashPath(pairs)

Complexity: O(d) DB reads
```

3.3 Database Key Schema

All nodes are stored as 32-byte SHA-256 hashes under string keys:

Key	Value	Purpose
<font name='Courier'>"name"	40 bytes: root (32) + depth (4 LE)	Tree metadata
<font name='Courier'>"name:level:pos"	32 bytes: SHA-256 hash	Node at (level, pos)

The metadata key is written atomically together with all path nodes using a single database batch — important for crash-consistency.

4. TypeScript Implementation

File: `eng-sessions/merkle-tree/src/merkle_tree.ts`

4.1 Constructor and Zero-Hash Pre-computation

```
constructor(private db: LevelUp, private name: string,
            private depth: number, root?: Buffer) {
  if (!(depth >= 1 && depth <= MAX_DEPTH)) throw Error('Bad depth');

  this.zeroHashes = this.computeZeroHashes(depth);

  if (root) {
    this.root = root;                  // restoring from DB
```

```
  } else {
    this.root = this.zeroHashes[depth]; // fresh empty tree
  }
}

private computeZeroHashes(depth: number): Buffer[] {
  const zeros: Buffer[] = new Array(depth + 1);
  zeros[0] = this.hasher.hash(Buffer.alloc(LEAF_BYTES));
  for (let i = 1; i <= depth; i++)
    zeros[i] = this.hasher.compress(zeros[i-1], zeros[i-1]);
  return zeros;
}
```

TypeScript: constructor initialises zero hashes and sets root

4.2 updateElement

```
async updateElement(index: number, value: Buffer) {
  const batch = this.db.batch();
  let currentHash = this.hasher.hash(value);
  let pos = index;

  for (let level = 0; level < this.depth; level++) {
    batch.put(this.nodeKey(level, pos), currentHash);

    const siblingPos = pos ^ 1;
    const sibling = await this.getNode(level, siblingPos);

    currentHash = (pos % 2 === 0)
      ? this.hasher.compress(currentHash, sibling)
      : this.hasher.compress(sibling, currentHash);

    pos = pos >> 1;
  }

  this.root = currentHash;
  this.writeMetaData(batch);
  await batch.write();
  return this.root;
}
```

TypeScript: updateElement — path traversal with batched writes

4.3 getHashPath

```
async getHashPath(index: number) {
  const pairs: Buffer[][] = [];

  for (let level = 0; level < this.depth; level++) {
    const pos      = index >> level;
    const leftPos  = pos & ~1;
    const rightPos = pos | 1;

    const left  = await this.getNode(level, leftPos);
    const right = await this.getNode(level, rightPos);

    pairs.push([left, right]);
```

```
  }

  return new HashPath(pairs);
}
```

TypeScript: getHashPath — returns sibling pairs from leaf to root

4.4 Test Results

#	Description	Expected Root	Pass
1	Empty depth-32 tree root	1c9a7e5f…	✓
2	Depth-2, 4 leaves, root + hash paths	e645e6b5…	✓
3	Depth-10, 128 inserts, restore DB	4b8404d0…	✓
4	Depth-32, 1024 inserts, path[100]	26996bfc…	✓

5. C++20 Implementation

File: `eng-sessions/merkle-tree-cpp/src/merkle_tree.hpp`

5.1 Constructor

```
MerkleTree(MockDB& db, const std::string& name,
           uint32_t depth, const sha256_hash_t& root = {})
    : db(db), name(name), depth(depth), root(root), hasher()
{
    if (!(depth >= 1 && depth <= MAX_DEPTH))
        throw std::runtime_error("Bad depth");

    // Pre-compute z_0 ... z_depth
    zero_hashes.resize(depth + 1);
    zero_hashes[0] = hasher.hash(std::vector<uint8_t>(LEAF_BYTES, 0));
    for (uint32_t i = 1; i <= depth; ++i)
        zero_hashes[i] = hasher.compress(zero_hashes[i-1], zero_hashes[i-1]);

    // Restore root from DB if present
    auto stored = db.get(name);
    if (stored.has_value()) {
        this->root = stored.value();
    } else {
        this->root = zero_hashes[depth];
        db.put(name, this->root);
    }
}
```

C++: constructor with DB restoration

5.2 update_element

```
sha256_hash_t update_element(uint64_t index,
```

```
                                const std::vector<uint8_t>& value)
{
    if (value.size() != LEAF_BYTES)
        throw std::runtime_error("Value must be exactly 64 bytes.");

    std::vector<MockDBBatchItem> batch;
    sha256_hash_t current_hash = hasher.hash(value);
    uint64_t pos = index;

    for (uint32_t level = 0; level < depth; ++level) {
        batch.push_back({ node_key(level, pos), current_hash });

        uint64_t sibling_pos = pos ^ uint64_t(1);
        sha256_hash_t sibling = get_node(level, sibling_pos);

        if (pos % 2 == 0)
            current_hash = hasher.compress(current_hash, sibling);
        else
            current_hash = hasher.compress(sibling, current_hash);

        pos >>= 1;
    }

    root = current_hash;
    batch.push_back({ name, root });
    db.batch_write(batch);
    return root;
}
```

C++: update_element — leaf update with batch write

5.3 get_hash_path

```
HashPath get_hash_path(uint64_t index) const
{
    std::vector<std::pair<sha256_hash_t, sha256_hash_t>> pairs;
    pairs.reserve(depth);

    for (uint32_t level = 0; level < depth; ++level) {
        uint64_t pos       = index >> level;
        uint64_t left_pos  = pos & ~uint64_t(1);
        uint64_t right_pos = pos | uint64_t(1);

        sha256_hash_t left  = get_node(level, left_pos);
        sha256_hash_t right = get_node(level, right_pos);

        pairs.push_back({ left, right });
    }

    return HashPath(pairs);
}
```

C++: get_hash_path — returns sibling pairs from leaf to root

5.4 Private Helper Methods

```
// Build the database key for node at (level, position)
std::string node_key(uint32_t level, uint64_t pos) const
{
    return name + ":" + std::to_string(level)
                + ":" + std::to_string(pos);
}

// Get node from DB, falling back to zero_hashes[level] if absent
sha256_hash_t get_node(uint32_t level, uint64_t pos) const
{
    auto stored = db.get(node_key(level, pos));
    if (stored.has_value()) return stored.value();
    return zero_hashes[level];
}
```

5.5 Test Results

#	Description	Expected Root	Pass
1	Empty depth-32 tree root	1c9a7e5f…	✓
2	Depth-2, 4 leaves, root + hash paths	e645e6b5…	✓
3	Depth-10, 128 inserts, restore DB	4b8404d0…	✓
4	Depth-32, 1024 inserts, path[100]	26996bfc…	✓

6. Topics Covered — What This Problem Tests

6.1 Core Data Structures & Algorithms

- **Binary trees and tree traversal**: leaf-to-root path traversal — identical pattern to B-tree node-splitting and red-black tree rotations.
- **Sparse data structures**: zero-hash placeholders avoid materialising 2^{32} nodes. Used in Ethereum's Patricia Merkle Trie, Solana's account tree, and Aztec's world state.
- **Hash functions as commitments**: SHA-256 is collision-resistant so modifying any leaf changes the root detectably.
- **Bit manipulation**: XOR for sibling (p ^ 1), AND-NOT for left child (p & ~1), OR for right child (p | 1), right-shift for level ascent (p >> k).
- **Amortised complexity**: O(d) operations per update, O(N·d) total for N insertions.

6.2 Cryptography Topics

- **SHA-256**: Merkle–Damgård construction, 64 rounds, 256-bit output. Collision resistance ensures distinct trees produce distinct roots.
- **Merkle trees in practice**: Bitcoin transaction Merkle roots, Ethereum state roots, certificate transparency (RFC 6962), IPFS CIDs, Git tree objects.

- **Merkle proofs**: O(log N) membership proof. A proof for a depth-32 tree is exactly 32 × 64 = 2,048 bytes.
- **Commitment schemes**: updating one leaf re-commits all ancestors in O(d) time — critical for rollup state transitions.
- **zkSNARK context**: the hash path is passed as a private witness into PLONK/Honk circuits. The circuit re-hashes it in-circuit to verify membership without revealing the leaf value.

6.3 Systems and Engineering Topics

- **Key–value persistence**: designing a key schema (`name:level:pos`) for O(1) lookup of any tree node.
- **Batched atomic writes**: grouping all path updates and metadata into one batch prevents partial writes on crash.
- **DB restoration**: serialising tree state (root + depth) so a new object can resume exactly where the previous one left off.
- **Async/await patterns** (TypeScript): LevelUp is Promise-based; all reads are awaited while writes are buffered in a batch.
- **C++20 features**: `std::optional` for nullable DB results, `std::array` for fixed-size hashes, `std::vector` for atomic writes.

6.4 Aztec-Specific Context

- **Note commitment tree**: sparse Merkle tree storing UTXO commitments; root is included in every Aztec block header.
- **Nullifier tree**: indexed Merkle tree preventing double-spend without revealing which note was spent (privacy-preserving).
- **Archive tree**: stores historical block headers, enabling proofs about past state.
- **Barretenberg / Noir circuits**: the getHashPath result feeds directly into zero-knowledge membership proofs compiled with Noir.

7. Complexity Summary

Operation	Time	Space
Constructor (zero hashes)	O(d) hash ops	O(d)
updateElement	O(d) hash ops + O(d) DB I/O	O(d)
getHashPath	O(d) DB reads	O(d)
getRoot	O(1)	O(1)
Total storage (N inserts)	O(N·d) writes	O(N·d) nodes

With d = 32: each updateElement call touches exactly 32 nodes on the path from leaf to root. Inserting 1,024 leaves (as in Test 4) performs 1,024 × 32 = 32,768 hash compressions and the same number of DB writes.

8. Conclusion

Both the TypeScript and C++ Merkle tree implementations pass all eight test cases. The four key design choices are:

- **Pre-computed zero hashes** in the constructor handle sparse nodes without any storage overhead.
- **String key schema** (`name:level:pos`) provides O(1) DB lookup for any node.
- **Single-loop traversal** from leaf to root for both writes (updateElement) and reads (getHashPath).
- **Atomic batch writes** ensure the tree state is always consistent, including the metadata root update.

✓ 4 TypeScript tests passing | ✓ 4 C++ tests passing | All expected hashes verified